

# 不要忘记

## 若vscode 没有C++环境

找到devC++的mingw 的bin文件夹，加入系统路径

再在vscode中 >edit ui

将路径输入，标准改为14

即可

## 运行代码

```
g++ A.cpp -o A
A
```

## 记得快读快输

```
ios::sync_with_stdio(0);
cin.tie(0), cout.tie(0);
```

## ！！！ 数据范围

注意爆int

long long: 8字节, -9,223,372,036,854,775,808到9,223,372,036,854,775,807

unsigned long long: 8字节, 0到18,446,744,073,709,551,615

如果爆long long 就用\_\_int128,但是注意输入输出

尽量不要使用double输入，容易超时

# \_\_int128的使用

```
#define int __int128
inline void read(int &n){
    int x=0,f=1;
    char ch=getchar();
    while(ch<'0' || ch>'9'){
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0' && ch<='9'){
        x=(x<<1)+(x<<3)+(ch^48);
        ch=getchar();
    }
    n=x*f;
}

inline void print(int n){
    if(n<0){
        putchar('-');
        n*=-1;
    }
    if(n>9) print(n/10);
    putchar(n % 10 + '0');
}

#undef int
```

## 快读模板

```
#define re register
#define il inline
il int read()
{
    re int x=0,f=1;char c=getchar();
    while(c<'0' || c>'9'){if(c=='-') f=-1;c=getchar();}
    while(c>='0' && c<='9') x=(x<<3)+(x<<1)+(c^48),c=getchar();
    return x*f;
}
```

# 随机数生成器

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());  
int Rand(int x, int y) {  
    return uniform_int_distribution<int>(x, y)(rng);  
}  
shuffle(a.begin(), a.end(), rng);
```

# 随机图

```
// n 为节点个数
// m 为边的个数，默认为 -1，表示建树
// root 为根节点编号，默认为 -1，
vector<array<int,2>> Graph(int n, int m=-1, int root=-1)
{
    assert(m== -1 || (n-1<=m && m<=1LL*n*(n-1)/2));
    assert(root== -1 || (1<=root && root<=n));
    vector<array<int,2>> E;
    set<array<int,2>> diff;
    // 先建一棵以 0 为根的树，然后选根偏移
    for (int i = 1; i < n; ++i) {
        E.push_back( { i, Rand(0, i-1) } );
    }
    if (root == -1) { root = Rand(0, n-1); }
    for (auto& [x, y]: E) {
        x = (x+root) % n + 1;
        y = (y+root) % n + 1;
        diff.insert( { x, y } );
        diff.insert( { y, x } ); // 无向图中需要添加这条反向边
    }
    // 从树建图
    if (m != -1) {
        for (int i = n-1; i < m; ++i) {
            while (1) {
                int x = Rand(1, n);
                int y = Rand(1, n);
                if (x == y || diff.count({x,y})) { continue; }
                E.push_back( { x, y } );
                diff.insert( { x, y } );
                diff.insert( { y, x } ); // 无向图中需要添加这条反向边
                break;
            }
        }
    }
    shuffle(E.begin(), E.end(), rng);
    return E;
}
```

# 对拍器

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int t=0;
    while(true){
        cout<<"test: "<<t++<<endl;
        system("data.exe>data.in");//data.exe 随机数生成器
        system("std.exe<data.in>data.out");//std.exe 标准程序
        system("solve.exe<data.in>solve.out");//solve.exe 暴力求解算法
        if(system("fc std.out solve.out>diff.log")){
            cout<<"WA"<<endl;
            break;
        }
    }
    cout<<"AC\n";
}
/*
// checker.cpp
while (1) {
    system("A_gen > _data.in"); // linux 下换成 ./xxx
    system("A_bf < _data.in > A.ans");
    system("A < _data.in > A.ans");
    if (system("fc A.out A.ans")) { // linux 下换成 diff
        cout << "WA\n"; return 0;
    } else {
        cout << "AC\n";
    }
}
}*/
```